

Разработка мобильных приложений

Лекция 1

Введение и практическое задание

Ключев А.О. к.т.н., доцент ФПИиКТ Университета ИТМО

Санкт-Петербург

2024

Контакты

- Ключев Аркадий Олегович
- E-mail: kluchev@yandex.ru
- ВК: <https://vk.com/aokluchev>
- Группа “РМП 2024 весна” в Telegram (см. сообщение в ИСУ)

Литература

На этом слайде представлены универсальные книги содержащие фундаментальные концепции, которые живут десятки лет. Концепции из этих книг пригодятся вам практически в любой области IT-технологий. По каждой отдельной теме будет специальная литература.

- Роберт Мартин. Чистая архитектура.
- Фредерик Брукс. Мифический человеко-месяц, или как создаются программные системы.
- Непейвода Н. Н. ,Скопин И. Н. Основания программирования.
- Гради Буч. Объектно-ориентированный анализ и проектирование с примерами приложений на C++
- Кстати, почему с книгами проблема и не имеет смысла покупать все книги подряд?

Литература по управлению проектами (для рабочих групп)

- Джеф Сазерленд. Scrum. Революционный метод управления проектами.
- Скрам. Описание
- Скрам Гайд. Исчерпывающее руководство по Скраму: Правила Игры
- Майк Кон. SCRUM. Гибкая разработка ПО.

В чем проблема университетов?

- Я посмотрел много роликов на YouTube, где успешные выпускники ругают свои университеты и вспомнил время, когда я был студентом. Вот основные претензии:
 - Старые, равнодушные и ленивые преподаватели, не желающие осваивать новые технологии
 - Заносчивые и высокомерные преподаватели, унижающие и оскорбляющие своих студентов
 - Некомпетентные преподаватели, которые на самом деле ничего не знают и не умеют, если их взять на работу
 - Устаревшее оборудование/программное обеспечение
 - Странные учебные программы, не совпадающие с реальностью индустрии IT технологий
 - Никому не нужные знания, отстающие от того, что используется в реальных проектах на пару десятков лет
 - Например, я изучал языки Pascal и Fortran, которым я ни разу в жизни не пользовался, а также ассемблер ЕС ЭВМ, от которого тоже не было никакого толка. Я изучал теорию, которую никто не мог применить на практике, потому что не знал как. Мне нужны были знания по архитектуре компьютеров и контроллеров, сетям, программной инженерии, современным ОС (я хотел заниматься Unix), языкам типа C/C++ и по микроконтроллерам, но этого не было в программе.
 - То, что мне реально требовалось на работе (на кафедре (!), а потом в фирме занимающейся разработкой промышленных контроллеров) я изучал сам, правда не без помощи хороших преподавателей кафедры, имеющих богатый практический опыт.
 - В результате ~~я прогулял большинство занятий~~ я закончил университет с хорошими оценками
- Вам нужно научиться как-то обходить подобные проблемы (если они появятся) и получать пользу от занятий, чтобы не терять свое время зря! **Помните, что не вас учат, а Вы учитесь!**

Что такое университет?

- Это фильтр. Вы обратили внимание, что вокруг вас в основном умные люди? Поступить в ИТМО довольно непросто.
- Концентрация специалистов. Университет это место, где сконцентрировано много интересных людей, обладающих разными знаниями, умениями, характерами, особенностями...
- Социализация. Вы привыкаете работать в коллективе. Вас готовят к тому, чтобы вы потом работали в корпорации или создали свой собственный бизнес.
- Полезные связи. Вы можете познакомиться со своими будущими коллегами или работодателями.
- Разные подходы и разные стили обучения. Примерно так работает N-версионное программирование. У каждого преподавателя свои способы до вас достучаться. Даже если какой-то курс вам сразу не понравился, вы все равно можете извлечь из этого для себя что-то полезное.
- Это путь. В процессе прохождения этого пути вы совершенствуете свой мозг, а диплом в конце обучения это доказательство полученных вами компетенций.
- Это короткий путь. Разные специалисты, представители бизнеса, индустрии и ваши друзья показывают вам как пройти путь обучения как можно быстрее и эффективнее.

Как учиться?

- Обучение = тренировка головного мозга
- По мере обучения вы совершенствуете свой мозг и решать профильные задачи становится все проще и проще. Чем больше практики, тем проще.
- Обучение в университете чем-то похоже на тренировку в спортзале. Только в одном случае вы тренируете свой мозг, а в другом тело и мышцы.
- Можно ли стать бодибилдером не занимаясь тяжестями, а просто сидя на диване и слушая лекции по бодибилдингу? Нет. Точно также нельзя стать инженером-программистом, слушая одни лекции по программированию. Вы должны тренировать свой мозг, а для этого вам нужно максимально задействовать свой мозг:
 - Много читать, смотреть видеоролики по профильным темам и **думать**
 - Решать множество технических задач, постепенно увеличивая их сложность.
- Мир технологий постоянно меняется, поэтому учиться вам придется всю жизнь.

Как закончить университет без какого-либо смысла

- Везде проскакивать по пути минимального сопротивления
- Не общаться с окружением
- Не участвовать в проектах
- Не делать докладов на конференциях и не писать статей
- Не участвовать в НИР
- Не работать по специальности
- Не читать дополнительную литературу
- Ничего не изучать по специальности
- Не развивать свой мозг
- ...
- Профит! Возможно вы получите свои тройки и получите корочку о высшем образовании, но как специалист вы будете так себе... Я бы такого не взял себе на работу.

Проблемы IT технологий

- Технологии быстро меняются. Как в таких условиях держать руку на пульсе современных IT технологий и быть востребованным на рынке труда?
- Стеки технологий создаются не великими учеными, а обычными программистами (со всеми вытекающими проблемами). Развитие технологий крайне многовекторно, идет эволюционным путем и оно довольно бессистемно. Победу той или иной технологии определяет успех на рынке, а не техническое совершенство и перспективы в будущем (например, см. историю взаимоотношений H. Вирта и фирмы Borland).
- Как не потерять работу из-за конкуренции с ИИ в ближайшем будущем?
- Какие языки программирования и стеки технологий учить?
- Какие книги читать?

Университет или курсы?

- Многие сейчас говорят, что в области IT технологий высшее образование не имеет никакого смысла
- Университет готовит инженеров, умеющих самостоятельно решать широкий спектр задач на базе имеющихся знаний или ученых (магистров, кандидатов и докторов наук), умеющих создавать новые знания.
- Курсы готовят «жрецов», умеющих пользоваться набором заклинаний, но не понимающих как они устроены (см. цикл романов (не сериал, а именно книги!) «Основание» Азимова).
- Если вам нужен специалист, с перспективой роста и работой на годы, то лучше брать на работу инженера. Если вам нужно сиюминутно заткнуть дыру в проекте с какой-то не особо интеллектуальной позицией, то подойдет «жрец» после курсов. Прелесть «жрецов» в том, что они легко взаимозаменяемы.
- Тем не менее курсы полезны (особенно бесплатные), так как они позволяют вам быстрее понять как сделать лабораторные работы.

Цель курса

- Показать подход к изучению нового стека технологий на примере экосистемы Android. Для этого мы собираемся:
 - Изучить основные концепции экосистемы Android (это базовые концепции, которые кочуют из проекта в проект годами и десятками лет без каких либо серьезных изменений).
 - Зная концепции, научиться пользоваться текущим стеком технологий экосистемы Android. Для этого необходимо научиться видеть концепции в описании стека технологий (это довольно непросто, так как устоявшейся терминологии как правило нет).
- Итак, мы хотим найти способ решения целого спектра разных технических задач в области вычислительной техники, требующих изучения неизвестного стека технологий на примере Android.

В чем состоит специфика обычных курсов по Android?

- Вас готовят к встраиванию в имеющийся производственный процесс (создание приложений на базе текущей версии Android), как правило для самых простых градаций (Junior).
- Вас учат решать шаблонные задачи
- Вам объясняют как решать определенный круг задач, но не объясняют почему именно так нужно решать задачи и как это все устроено внутри. Решая задачу вы скорее всего не сможете объяснить никому что вы делаете. Действие напоминает заклинания из волшебной книги, которые вы запомнили, но не можете объяснить. Еще этот процесс можно сравнить с китайской комнатой.
- При появлении нового стека технологий вам придется снова идти на курсы.

В чем ваше преимущество как студентов?

- В вашей рабочей группе командуете вы, вы никому ничего не должны и ваш проект ограничивается только вашей фантазией, семестром и физическими возможностями
- Вы можете посмотреть на разные подходы к программированию, изучить новые стеки технологий (и попытаться понять как это делать эффективнее в следующий раз) и изучить то, что вам кажется полезным и интересным
- При желании можно расширить рамки проекта

Курс «разработка мобильных приложений»

- Два варианта заданий:
 - Индивидуальное задание - приложение (Android)
 - Групповое задание – приложение (Android) + бэкэнд (группа до 5-6 человек)
- Отчет в любом из вариантов задания должен содержать 4 основных раздела + оглавление, введение, заключение и список литературы:
 1. Техническое задание
 2. Описание архитектуры
 3. Описание реализации
 4. Описание системы тестирования
- Для выполнения лабораторных работ желательно иметь с собой ноутбук с Android Studio
- Если нет подходящего телефона с Android можно использовать симулятор.

Про лекции в курсе

- В настоящий момент я не зарабатываю денег на проектировании мобильных приложений и я не буду изображать глубокого специалиста.
- Я не буду углубляться в тонкости конкретных версий Android по целому ряду причин:
 - Объем этих знаний огромен, он не уложится в короткий курс;
 - Я этими знаниями не владею в должной мере (я всегда разбирался с тонкостями в рабочем порядке, а потом благополучно забывал все, так как голова не резиновая!);
 - Изучать тонкости и лезть слишком глубоко в технологии впродолжение нет смысла, так как пока вы узнаете эти знания перестанут быть актуальными и без практики вы скорее всего все быстро забудете.
- **Но как нам тогда быть?**
- **Наша задача проста. На примере данной вам задачи вы должны выработать навык получения новых знаний о конкретных технологиях и научиться быстро этими технологиями пользоваться на практике.**

Контрольные работы

- У нас по плану 5 контрольных, но они будут на каждой лекции (до 8 штук) для мотивации посещения лекций и закрепления материала предыдущей лекции
- Тема контрольной берется из предыдущей лекции, как часть вашего отчета по лабораторной работе или по всему пройденному материалу (если контрольная рубежная).
- Обычно 3 вопроса
- Ответ в свободной форме

Что мы вообще делаем?

- Мы имитируем либо одинокого разработчика-фрилансера, либо небольшой стартап и пытаемся сделать минимально жизнеспособный вариант приложения (MVP) с документацией за этот семестр.
- Я понимаю, что у вас куча других предметов, вы работаете, устали и вообще у вас лапки. Поэтому программы могут быть простыми, им разрешается иногда падать, а в документации допускаются пробелы.
- Но, вам предстоит работать и я надеюсь, что в этом курсе вы узнаете что-то полезное для себя.

Выбор операционной системы для работы

- Желательно иметь на своем ноутбуке Linux (единственной или второй системой). Я предпочитаю Ubuntu или Linux Mint.
- В мире разработчиков софта популярны Mac OSx и Linux
- Linux удобнее, если вам потребуется запускать какие-то серверные приложения. Большинство серверов и СУБД ставятся одним кликом из репозитория. Более того, собрать Android из исходников можно только в Linux (вдруг кому-то захочется получить свой Andoid).
- В Mac OSx это делать обычно сложнее чем в Linux, есть проблемы с виртуальными машинами на ноутбуках с процессорами M1/M2. Зато эргономика Macbook почти идеальна и никто не мешает использовать docker контейнеры.
- В Windows я запускаю Linux в виртуальной машине, использую docker контейнеры или Windows System for Linux (WSL, WSL2). WSL/WSL2 работает на Windows 10+

Стек технологий

- IDE - IntelliJ IDEA
 - Android Studio
 - PyCharm (Python)
- Язык программирования (Android и бэкэнд) - Kotlin
- Брокер сообщений для бэкэнд - Redis Server (самый простой вариант)
- СУБД для бэкэнд - PostgreSQL
- Хранение больших объемов данных и быстрая выдача клиенту – Clickhouse DB
<https://clickhouse.com/docs/ru>
- Система контроля версий - Git (GitHub, GitLab, GitFlic и т.п.)
- Документация в исходных текстах (Dokka) + Markdown (Obsidian)
- Отчет - PDF (с оглавлением, введением, заключением, списком литературы и красивыми картинками, оформленный по ГОСТ, почти как диплом)

Тема проекта – умный дом

- В этом семестре вы должны разработать систему «Умный дом»
- Два варианта:
 - Индивидуальная работа – делается только мобильное приложение для Android
 - Групповая работа для группы 5-6 человек, делается приложение и бэкэнд

Зачем нужна система «Умный дом»?

- Комфорт
 - Автоматизации рутинных операций. Например, выставление температуры во всех комнатах, управление группами светильников, переключение режимов работы (проживание, консервация дома, охрана и т.п).
 - Комфортный климатический режим. Например, разогрев загородного дома к приезду, автоматическая поддержка заданной температуры и влажности.
 - Комфортный режим освещения. Автоматическое управление уличным и домашним освещением без участия человека.
 - Контроль за качеством воздуха, управление вентиляцией
- Энергосбережение
 - В загородных домах зимой тратятся существенные суммы на отопление, а летом на кондиционирование воздуха.
- Безопасность
 - Сигнализация и имитация присутствия
 - Видеонаблюдение
 - Защита от протечек
 - Пожарная сигнализация

Стек технологий

- Мобильное приложение (групповые и индивидуальные проекты):
 - Android (реальное устройство или симулятор)
 - Kotlin
 - Jetpack compose
- Бекэнд (групповые проекты)
 - *nux: Linux (WSL), MacOS
 - Kotlin, Ktor, корутины, библиотека для сериализации JSON и т.п.
 - Redis Server – кэш БД, брокер сообщений
 - PostgreSQL
 - ClickHouse
- За отклонение от этого списка буду снижать балл (все как в реальных проектах, только в реальных проектах вы просто не сдадите работу заказчику, если ваша работа не будет соответствовать техническому заданию)

Требования к системе умный дом

- Для индивидуального проекта
 - Имитация классического приложения для системы умный дом
- Для группового проекта
 - Создание системы для обслуживания 10-ти тысяч домов
 - Состав системы
 - Мобильное приложение (личный кабинет и управление умным домом)
 - Микросервис для связи с мобильными приложениями
 - Микросервис для работы с БД
 - Микросервис для ведения логов
 - Микросервис для имитации 10-ти тысяч домов (климат, электрооборудование, люди, погода и т.п.)
 - Микросервис имитатор 10-ти тысяч мобильных приложений для тестирования системы (без GUI, просто рандомные запросы через API к серверу).
- Функции системы
 - Управление освещением, климатом, сигнализация, автоматизация рутинных операций, интеграция с голосовым помощником и инфраструктурой Яндекс (по желанию)

Зачем нужна рабочая группа?

- Вам дается попытка сделать свой проект в коллективе (это непросто)
- Сложность проекта можно увеличить до размера, похожего на реальный проект
- Сложность проекта достаточно велика, чтобы в нем появилась необходимость в грамотном проектировании кода и в архитектуре.

Почему в рамках курса по разработке мобильных приложений будет какой-то бекэнд?

- В настоящее время оторванных от сети Интернет мобильных приложений очень мало и это что-то простое, типа калькулятора
- Мобильная разработка не очень сложна, как правило весь функционал расположен в облаке, а в мобильном приложении есть только интерфейс пользователя
- Мобильные технологии очень быстро меняются, нет смысла углубляться в тонкости, так как через пару лет все будет иначе.
- Если вы хотите сконцентрировать свои усилия только мобильном приложении, то вы можете делать индивидуальное задание

Зачем нужен отчет?

- На самом деле это не совсем отчет, это так называемая «пояснительная записка», то есть документ являющийся описанием проекта.
- Практика показывает, что большинство студентов (по крайней мере среди тех, кого я брал на работу) не умеет писать документацию, что снижает качество проектов и замедляет карьерный рост сотрудников. Умение формулировать мысли говорит о том, что у человека порядок в голове и косвенно подтверждает гипотезу о том, что в проекте у такого человека тоже все более-менее нормально и он понимает, что он делает.

Оформление отчета

- Титульный лист
- Оглавление
- Введение
- 1. Техническое задание
- 2. Архитектура системы
- 3. Описание реализации
- 4. Тестирование
- Заключение
- Список литературы
- Приложения

Оформление текста

- Как в дипломе, пока нет особо жестких требований. Скачайте методичку про оформление диплома, например эту:
 - Калинина М.И., Смирнов С.Б. Методические указания по выполнению выпускных квалификационных работ для бакалавров - Санкт-Петербург: Университет ИТМО, 2016. - 40 с. - экз.
- Государственные стандарты (ГОСТы). Посмотрите, лучше привыкнуть к правильному оформлению документации сразу:
 - ГОСТ 2.105-95 «Единая конструкторская документация. Общие требования к текстовым документам».
 - ГОСТ 7.32-2017 «Межгосударственный стандарт. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления».

Титульный лист

- Название проекта
 - ФИО автора или авторов
 - Руководитель
 - Организация (в нашем случае Университет ИТМО)
 - Год
-
- Есть стандартные формы титульного листа

Оглавление

- Зачем нужно? Оглавление позволяет быстро оценить структуру документа и прочитать только те разделы, которые реально полезны в работе.
- Стоимость создания оглавления низка, а польза очень высока.
- Для создания оглавления можно воспользоваться любым вордо-подобным редактором (Мой офис, Яндекс 360 и т.п.) и создать там оглавление. Оглавление создается автоматически, если вы будете использовать в названиях разделов стили «Заголовок 1», «Заголовок 2» и т.д.
- Количество уровней заголовков лучше не делать больше трех, так как большее количество уровней плохо воспринимается при чтении таких маленьких документов.

Колонтитулы и номер страниц

- Внизу страницы (за исключением титульного листа) нужно вставить номер страницы. Исключительно полезная штука, если у вас документ распечатан и вам нужно найти нужный раздел. В чисто электронном документе есть гиперссылки, но все равно номера страниц крайне полезны.

Введение

- Введение дает читателю возможность быстро разобраться о назначении данного документа. Нужно написать:
 - Актуальность работы
 - Цели и задачи работы
- Актуальность – аргументы доказывающие важность, нужность, своевременность и необходимость данной разработки.
- Цель работы – получение чего-либо полезного. Например, наша цель - согреться. Задачи – последовательность действий для достижения нашей цели. В случае с целью «согреться», нам нужно решить следующие задачи:
 - Найти сухие ветки
 - Сложить костер
 - Найти спички, зажигалку или каким либо иным способом получить огонь.
 - Разжечь костер.

Техническое задание

- Техническое задание (ТЗ) — перечень требований, условий, целей, задач, **поставленных заказчиком** (*) в письменном виде, документально оформленных и выданных исполнителю работ проектно-исследовательского характера. Такое задание обычно предшествует разработке проектов и призвано ориентировать разработчика на создание проекта, удовлетворяющего желаниям заказчика и соответствующего условиям использования, применения разрабатываемого проекта, а также ресурсным ограничениям.
- (*) *На самом деле ТЗ пишет исполнитель в 99% случаев*
- За основу можно взять ГОСТ 19.201-78 «Техническое задание, требования к содержанию и оформлению»

Разделы ТЗ в вашем отчете

- Введение;
- Назначение разработки;
- Требования к программе или программному изделию;
 - Функции системы
 - Состав системы
 - Характеристики системы

Как писать техническое задание?

- Составляете список заинтересованных лиц (например, мы делаем какую-то систему для какого-то завода)
 - Директор завода
 - Мастер цеха
 - Бухгалтер
 - Главный инженер
 - Рабочий
 - Охранник
 - Дворник
 - Гость/посетитель
 - Разработчик системы
- Составляете список важных аспектов разработки (инструментальный, прикладной, надежность, безопасность, энергопотребление (мобильная разработка!) и т.п.)
- Составляете список требований для каждого из заинтересованных лиц
- Оцениваете сроки, бюджет разработки и возможности разработчиков
- Формулируете ТЗ, которое вы можете успешно выполнить и которое удовлетворит заказчика.

Архитектура проекта

- Архитектура проекта позволяет понять как устроен проект внутри, из каких частей состоит, как эти части друг с другом связаны и как они друг с другом взаимодействуют.
- Для кого нужна архитектура?
 - Для технических руководителей.
 - Для архитекторов проектов.
 - Для старших разработчиков.
- Насколько глубоко нужно погружаться в архитектуру? Погружаться нужно до уровня, достаточного для создания спецификаций для загрузки работой рядовых программистов.
- Можно ли давать архитектуру в качестве технического задания для рядовых исполнителей? Нет, в большинстве случаев это бесполезно, так как при «вхождении в IT» на разных курсах по программированию таким высоким материям не учат.
- Архитектура вашего проекта это сложный и ответственный раздел, которому нужно посвящать много времени.

Описание реализации

- Описание реализации нужно для того, чтобы у участников проекта было понимание тонкостей реализации классов, методов и структур данных.
- По собственному опыту: при интенсивной и разноплановой работе над несколькими проектами понимание собственных исходных текстов может исчезнуть за пару месяцев.
- Как нужно документировать?
 - Желательно без отрыва от исходных текстов проекта. Все благие намерения по написанию документации в отдельном и красивом документе как правило заканчиваются тем, что из-за большой нагрузки документ забрасывается и очень быстро теряет актуальность.

Виды тестирования



Раздел про тестирование

- Мы будем использовать только три вида тестов:
 - Модульное (юнит-тесты отдельных классов, механизм тестирования встроен в IntelliJ IDEA)
 - Интеграционное (тестирование подсистем, микросервисов, библиотек,...)
 - Системное (все целиком)
- Для интеграционного и системного тестирования вам понадобится разработать тестовое инструментальное обеспечение на языке Kotlin
 - Имитатор бэкэнда в индивидуальных проектах
 - Средства для тестирования микросервисов в проектах с рабочей группой

Список литературы

- Вы не можете что либо написать не делая ссылок на источники по следующим причинам:
 - Нет смысла повторять то, что уже написано другими. Достаточно сослаться на статью или книгу.
 - Цитируя чужие тексты без указания источника вы занимаетесь плагиатом. Например, с дипломом такое цитирование закончится тем, что вы не пройдете через систему «Антиплагиат». Аналогичная проблема может возникнуть при написании статьи или книги.
 - Читателю может понадобиться дополнительная информация из первоисточника, который вы указали.
 - Читателю может понадобиться доказательство ваших утверждений.
- ГОСТ Р 7.0.80—2023 Система стандартов по информации, библиотечному и издательскому делу БИБЛИОГРАФИЧЕСКАЯ ЗАПИСЬ

Примеры библиографических записей

- Лаврищева, Е. М. Программная инженерия и технологии программирования сложных систем : учебник для вузов / Е. М. Лаврищева. — 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2023. — 432 с. — (Высшее образование). — ISBN 978-5-534-07604-2. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: <https://urait.ru/bcode/513067> (дата обращения: 10.09.2023).. Общие требования и правила составления
- Исследовано в России [Электронный ресурс]: многопредмет. науч. журн. / Моск. физ.-техн. ин-т. — Электрон. журн. — Долгопрудный: МФТИ, 1998. — режим доступа к журн.: <http://zhurnul.milt.rissi.ru> (дата обращения: 06.05.2023)
- Lindell I. V., Sihvola A. H., Tretyakov S. A., Viitaten A. J. Electromagnetic Waves in Chiral and Bianisotropic Media. Boston — London: Artech House, 1994.

Заключение

- В заключении приводятся результаты и достижения, которые вы получили в процессе работы над проектом.
- Результат это обычно выполненная задача, указанная в начале документа. Например:
 - Собраны сухие ветки в достаточном количестве.
 - Получен огонь с помощью куска кремня и рессоры от тойоты.
 - Разведен костер.
 - Работа с костром показала, следующее:
 - Спать зимой на снегу вокруг костра неудобно, так как спереди жарко, а сзади холодно.
 - Волки боятся огня, но они громко воют и страшно смотрят светящимися глазами из темноты.
 - Запас дров нужно рассчитывать таким образом, чтобы с одной стороны не слишком утомить себя заготовкой, а с другой, чтобы хватило дров для поддержания горения до утра.

Требования к исходным текстам

- Исходные тексты пишутся на языке с использованием соответствующих конвенций:
 - Coding conventions - <https://kotlinlang.org/docs/coding-conventions.html>
- В IntelliJ IDEA есть опция, позволяющая отформатировать код правильно.
- Комментировать код нужно тех местах, где это необходимо.
 - Для Kotlin используем Dokka - <https://kotlinlang.org/docs/kotlin-doc.html>

Спасибо за внимание!